

Appunti di Information Retrieval

Anno Accademico 2026/2027

Indice

1	Introduzione e Architettura dell'Information Retrieval	1
1.1	Definizione e Motivazioni	1
1.1.1	Information Retrieval vs. Data Retrieval (Basi di Dati)	2
1.2	I Due Macro-Obiettivi: Efficacia ed Efficienza	3
1.2.1	Efficienza e Architettura Hardware del Calcolatore	3
1.3	Architettura di un Sistema di Information Retrieval	4
1.3.1	Pipeline di Text Processing	4
1.4	Proprietà del Linguaggio e Modelli di Reperimento	6
1.5	L'Inverted Index (Indice Invertito)	6
1.5.1	Elaborazione delle Query e Algoritmo di Merge	7
1.6	Tecniche di Compressione dell'Indice	8
1.6.1	Binary Interpolative Coding	8
1.6.2	Compressione Basata su Dizionario: LZ77	9
1.7	Evoluzione verso il Neural Information Retrieval	10
1.7.1	Le Tre Famiglie di Rappresentazioni Vettoriali	11
1.8	Ricerca dei Vicini Approssimati (k -ANN)	12
2	Valutazione dei Sistemi di Information Retrieval	13
2.1	Come Valutare la Qualità di un Motore di Ricerca	13
2.2	Misurare la Soddisfazione dell'Utente	14
2.3	Il Paradigma di Cranfield e le Collezioni Benchmark	15
2.3.1	La Sfida della Scalabilità Combinatoria	15
2.3.2	Crowdsourcing: Opportunità e Limiti	16
2.3.3	L'Infrastruttura TREC (<i>Text REtrieval Conference</i>)	16
2.3.4	La Tecnica del Pooling	16
2.4	Bisogno Informativo vs. Query	17
2.5	Metriche di Valutazione Non Ordinate (<i>Unranked Measures</i>)	18
2.5.1	Precisione, Richiamo e loro Trade-off	19
2.5.2	La Media Armonica: F-Measure (F_1 -Score)	19
2.5.3	La Misura F_β Pesata	20

2.6	Metriche Basate sul Ranking (<i>Ranked Measures</i> - Rilevanza Binaria)	20
2.6.1	Precision@k ($P@k$) e Recall@k ($R@k$)	21
2.6.2	Average Precision (AP)	21
2.6.3	Mean Average Precision (MAP)	22
2.6.4	Mean Reciprocal Rank (MRR)	22
2.7	Oltre la Rilevanza Binaria: Rilevanza Graduata e NDCG . . .	23
2.7.1	Cumulative Gain (CG) e Discounted Cumulative Gain (DCG)	23
2.7.2	Normalized Discounted Cumulative Gain (NDCG) . . .	24
2.7.3	Esempio Numerico Completo: Calcolo Passo-Passo . .	24
2.7.4	Confronto tra Funzioni di Ranking Alternative	25

Capitolo 1

Introduzione e Architettura dell'Information Retrieval

1.1 Definizione e Motivazioni

L'**Information Retrieval** (IR, *Reperimento delle Informazioni*) è la disciplina scientifica e ingegneristica che studia la rappresentazione, la memorizzazione, l'organizzazione e l'accesso a collezioni di dati prevalentemente non strutturati o semi-strutturati, al fine di soddisfare un **bisogno informativo** (*information need*) espresso da un utente [1, 2].

Definizione 1.1.1 (Information Retrieval (Manning et al.)). L'Information Retrieval consiste nel reperire materiale di natura prevalentemente non strutturata (solitamente documenti testuali) che soddisfa un bisogno informativo all'interno di collezioni di grandi dimensioni memorizzate su calcolatori elettronici.

Nell'ecosistema tecnologico contemporaneo, l'Information Retrieval non si limita ai soli motori di ricerca Web generalisti (come Google o Bing), ma costituisce l'infrastruttura abilitante di:

- **E-commerce e marketplace:** reperimento di prodotti basato su descrizioni testuali, attributi e recensioni degli acquirenti.
- **Enterprise e Desktop Search:** indicizzazione e ricerca semantica su repository aziendali (documenti, email, codebase sorgente).
- **Sistemi RAG (*Retrieval-Augmented Generation*):** alimentazione di modelli linguistici di grandi dimensioni (LLM) con documenti esterni contestualmente rilevanti per azzerare le allucinazioni e garantire informazioni aggiornate.

- **Agenti autonomi e motori di raccomandazione:** filtraggio collaborativo e basato su contenuto per la scoperta personalizzata di contenuti.

1.1.1 Information Retrieval vs. Data Retrieval (Basi di Dati)

Una distinzione metodologica basilare risiede nella contrapposizione tra il modello convenzionale dei database relazionali (*Data Retrieval*) e l'Information Retrieval:

Dimensione	Data Retrieval (Database SQL)	Information Retrieval
Struttura dei Dati	Dati altamente strutturati con schema relazionale rigido (tabelle, tipi, vincoli di integrità)	Dati non strutturati o semi-strutturati (testo libero, HTML, PDF, metadati)
Linguaggio di Query	Query formali e deterministiche (ad es. SQL) con semantica algebrica esatta	Query in linguaggio naturale o parole chiave (spesso ambigue, brevi e incomplete)
Criterio di Matching	Corrispondenza booleana esatta: un record appartiene o non appartiene al risultato	Corrispondenza parziale e probabilistica: i documenti presentano gradi variabili di rilevanza
Ordinamento Risultati	Nessun ordinamento intrinseco (se non specificato con ORDER BY)	Ordinamento fondamentale basato sulla stima di rilevanza (<i>Relevance Ranking</i>)
Tolleranza all'Errore	Nulla: errori di sintassi o valori inattesi generano errori o insiemi vuoti	Elevata: tolleranza a refusi, sinonimia, polisemia e variabilità morfologica

Tabella 1.1: Confronto analitico tra Data Retrieval tradizionale e Information Retrieval.

La Rilevanza: Il Cardine dell'IR

Mentre in una base di dati un record soddisfa o viola in modo booleano un predicato logico, nell'Information Retrieval il fulcro è la stima del grado di **rilevanza** rispetto al bisogno informativo cognitivo dell'utente. La rilevanza è soggettiva, situata nel contesto operativo ed evolutiva nel tempo.

1.2 I Due Macro-Obiettivi: Efficacia ed Efficienza

Un sistema di reperimento dell'informazione su larga scala deve simultaneamente perseguire due obiettivi spesso in contrasto:

1. **Efficacia (*Effectiveness*):** Quantifica la qualità intrinseca dei risultati restituiti, misurando quanto accuratamente il ranking soddisfa il bisogno informativo dell'utente (argomento approfondito nel Capitolo 2).
2. **Efficienza (*Efficiency*):** Quantifica il costo computazionale, il throughput (query al secondo) e la latenza di risposta (generalmente richiesta nell'ordine dei 10–50 millisecondi per query su archivi di decine di miliardi di documenti).

1.2.1 Efficienza e Architettura Hardware del Calcolatore

L'efficienza di un motore di ricerca moderno non dipende unicamente dalla complessità asintotica degli algoritmi, ma dall'interazione profonda con l'architettura dei processori e della memoria:

- **Pipelining e Processori Superscalari:** I moderni processori suddividono l'esecuzione delle istruzioni in stadi sequenziali (Instruction Fetch, Decode, Execute, Memory Access, Write Back) ed eseguono molteplici istruzioni per ciclo di clock.
- **Pipeline Hazards (Rischi di Pipeline):**
 - *Structural Hazards:* Conflitti nell'uso simultaneo di risorse hardware limitate (ad es. unità di memoria o registri).
 - *Data Hazards:* Dipendenze logiche tra istruzioni consecutive (RAW: Read-After-Write), che costringono il processore a inserire cicli di stallo (*pipeline bubble*) se i dati non sono pronti.

- *Control Hazards*: Salto condizionale non previsto. Nella scansione e nell'intersezione delle posting list, un'istruzione condizionale errata (*branch misprediction*) svuota la pipeline, comportando una penalità di 15–20 cicli di clock per ogni errore di predizione.
- **Gerarchia di Memoria (*Memory Hierarchy*)**: Il tempo di accesso ai registri della CPU è sub-nanosecondo, alle cache L1/L2/L3 è di pochi cicli, mentre l'accesso alla RAM DDR richiede 50–100 ns e lo storage secondario (NVMe/SSD) richiede decine di microsecondi. La progettazione delle posting list privilegia layout ad accessi sequenziali contigui per massimizzare il riuso delle linee di cache e sfruttare l'hardware prefetcher.
- **Vettorizzazione SIMD (*Single Instruction Multiple Data*)**: L'impiego di registri vettoriali a 128, 256 o 512 bit (AVX-2, AVX-512, ARM NEON) consente di confrontare 8 o 16 identificatori di documento con una singola istruzione macchina, accelerando drasticamente gli algoritmi di intersezione.
- **Calcolo Parallelo e Sharding Distribuito**: Gli indici che superano la capacità di un singolo nodo vengono frammentati (*sharded*), tipicamente per partizionamento documentale (*Document Sharding*), consentendo a cluster di servire migliaia di query parallele aggregando le risposte mediante broker centrali.

1.3 Architettura di un Sistema di Information Retrieval

Come illustrato nello schema di Figura 1.1, un sistema IR adotta una rigorosa separazione tra la **Pipeline Offline** (indicizzazione e addestramento) e la **Pipeline Online** (interrogazione a bassa latenza).

1.3.1 Pipeline di Text Processing

Prima dell'indicizzazione, ogni flusso testuale attraversa quattro fasi sequenziali:

1. **Tokenizzazione (*Tokenization*)**: Segmentazione della stringa di caratteri in sequenze di token lessicali, gestendo separatori, acronimi e punteggiatura.

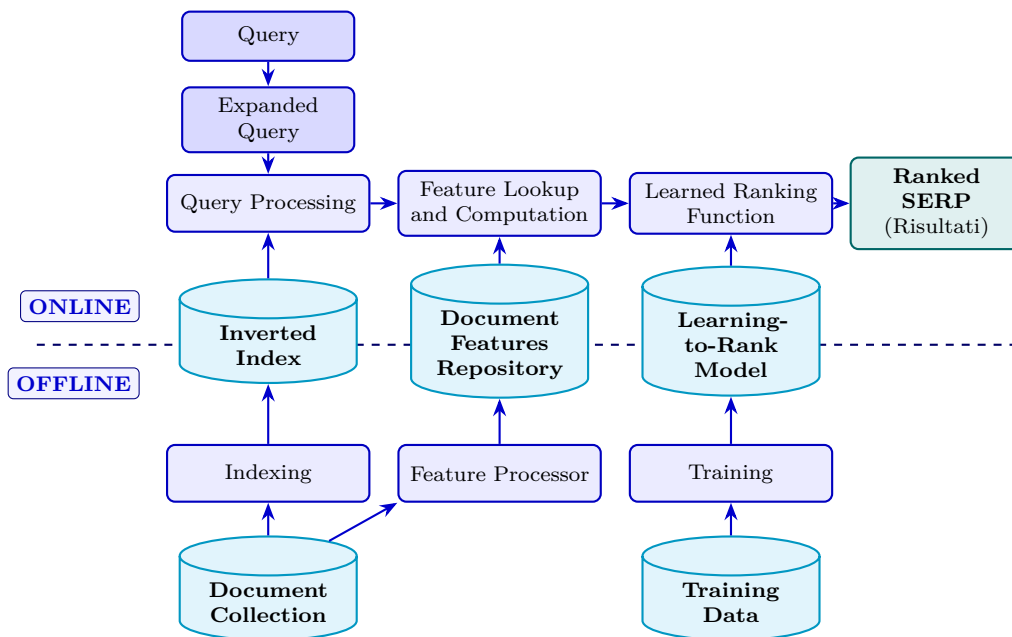


Figura 1.1: Architettura a griglia di un moderno motore di ricerca (pipeline Online vs. Offline).

2. **Normalizzazione e Case Folding:** Conversione a caratteri minuscoli e rimozione o armonizzazione di diacritici (es. accenti) per garantire la coerenza tra query e documenti.
3. **Filtraggio delle Stop Words:** Eliminazione di vocaboli ad altissima frequenza ma privi di valore semantico discriminante (articoli, preposizioni, congiunzioni).
4. **Stemming e Lemmatizzazione:**
 - *Stemming:* Procedura algoritmica basata su regole euristiche (ad es. l'algoritmo di Porter [1]) che tronca suffissi morfologici per estrarre la radice (*stem*). Esempio: "searching", "searched" → "search".
 - *Lemmatizzazione:* Analisi linguistica formale che si avvale di dizionari morfologici per mappare ogni forma flessa al suo lemma canonico. Esempio: forme verbali "sono", "fui" → lemma "essere".

1.4 Proprietà del Linguaggio e Modelli di Reperimento

La distribuzione delle parole nei testi naturali segue leggi empiriche che condizionano sia la progettazione degli indici sia le formule di ranking:

- **Legge di Zipf:** La frequenza $f(r)$ di un termine è inversamente proporzionale al suo rango r nella graduatoria di frequenza:

$$f(r) \propto \frac{1}{r} \quad (1.1)$$

Poche parole frequentissime occupano la maggioranza delle occorrenze, mentre la stragrande maggioranza dei termini compare pochissime volte (*long tail*).

- **Legge di Heaps:** La dimensione del vocabolario $|V|$ cresce come legge di potenza rispetto al numero totale di parole N della collezione:

$$|V| = k \cdot N^\beta \quad (\text{con } 0.4 \leq \beta \leq 0.6) \quad (1.2)$$

Il vocabolario non si satura mai completamente all'aumentare dei documenti.

I principali modelli di reperimento includono:

1. **Modello Booleano:** Match deterministico basato su operatori logici ($\wedge, \vee, \neg$), privo di punteggio intrinseco di rilevanza.
2. **Vector Space Model (TF-IDF):** Rappresentazione di query e documenti come vettori in uno spazio dimensionale $|V|$, pesati con frequenza del termine (TF) e frequenza inversa nei documenti (IDF). La rilevanza è calcolata tramite la similitudine del coseno.
3. **Modello Probabilistico (BM25):** Standard de facto del reperimento lessicale basato sul principio del 2-Poisson, che include saturazione non lineare del TF e normalizzazione della lunghezza del documento [2].

1.5 L'Inverted Index (Indice Invertito)

L'**Inverted Index** inverte l'orientamento logico della memorizzazione documentale, realizzando una mappatura $t \mapsto \mathcal{P}_t$, dove $\mathcal{P}_t$ è la lista ordinata dei documenti contenenti il termine t [1].

Definizione 1.5.1 (Inverted Index). Un Inverted Index è costituito da due componenti:

1. **Dizionario (*Dictionary* o *Vocabulary*)**: Contiene l'insieme ordinato di tutti i termini unici presenti nella collezione, memorizzato con strutture di lookup rapido (B-Tree o Hash Table), insieme alla frequenza documentale df_t e al puntatore all'inizio della lista.
2. **Posting List**: Per ogni termine t , un array ordinato per identificatore di documento crescente ($DocID_1 < DocID_2 < \dots$). Ciascun elemento (posting) può includere metadati accessori quali frequenza locale del termine (tf) e offset posizionali all'interno del documento.

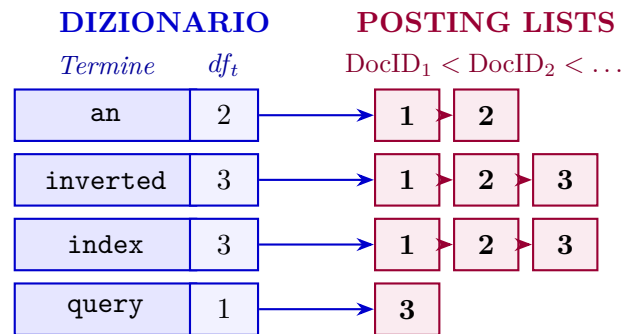


Figura 1.2: Struttura di un Inverted Index: Dizionario dei termini con frequenza documentale df_t e Posting Lists concatenate per DocID crescente.

1.5.1 Elaborazione delle Query e Algoritmo di Merge

L'ordinamento intrinseco dei DocID consente di risolvere le query congiuntive (**AND**) mediante una scansione lineare sincronizzata a due puntatori (Algoritmo 1).

Algorithm 1: Merge Intersection per Query Congiuntive ($P_1 \wedge P_2$)

Dati: Due posting list ordinate P_1 e P_2

Risultato: Lista ordinata R dei documenti presenti in entrambe le liste

```
1  $R \leftarrow \emptyset$ 
2  $i \leftarrow 1; j \leftarrow 1$ 
3 while  $i \leq |P_1|$  e  $j \leq |P_2|$  do
4   if  $P_1[i] == P_2[j]$  then
5      $R.append(P_1[i])$ 
6      $i \leftarrow i + 1$ 
7      $j \leftarrow j + 1$ 
8   else if  $P_1[i] < P_2[j]$  then
9      $i \leftarrow i + 1$ 
10  else
11     $j \leftarrow j + 1$ 
12 return  $R$ 
```

La complessità temporale nel caso peggiore è $\mathcal{O}(|P_1| + |P_2|)$. Nelle interrogazioni disgiuntive (**OR**), il merge include l'unione dei DocID preservando l'ordinamento. Nelle ricerche per frase (*Phrase Search*), l'algoritmo verifica che la differenza tra gli offset posizionali corrisponda esattamente alla distanza tra le parole nella query.

1.6 Tecniche di Compressione dell'Indice

La compressione dell'Inverted Index risponde a una duplice esigenza: ridurre drasticamente l'occupazione di memoria secondaria e RAM, e convertire un collo di bottiglia di trasferimento I/O (lento) in un carico computazionale di decompressione da parte della CPU (estremamente veloce).

1.6.1 Binary Interpolative Coding

Il **Binary Interpolative Coding** (proposto da Moffat e Stuiver [3]) è una tecnica di compressione specificamente concepita per sequenze strettamente crescenti di interi (come i DocID nelle posting list).

Definizione 1.6.1 (Principio di Interpolazione). Data una sottolista ordinata di n interi $L = [x_1, x_2, \dots, x_n]$ nota per essere compresa nell'intervallo $[l, h]$ (ovvero $l \leq x_1 < x_2 < \dots < x_n \leq h$), si seleziona l'elemento centrale $m = \lfloor (n+1)/2 \rfloor$. Poiché devono esistere almeno $m-1$ valori distinti prima

di x_m e almeno $n - m$ valori distinti dopo x_m , il valore x_m è strettamente vincolato nell'intervallo ristretto:

$$l + m - 1 \leq x_m \leq h - n + m \quad (1.3)$$

Il numero di possibili valori distinti per x_m è dato da:

$$R = (h - n + m) - (l + m - 1) + 1 = h - l - n + 1 \quad (1.4)$$

Il valore $(x_m - (l + m - 1))$ viene quindi codificato in binario utilizzando esattamente $\lceil \log_2 R \rceil$ bit. Successivamente, l'algoritmo si applica ricorsivamente alla metà sinistra rispetto a $[l, x_m - 1]$ e alla metà destra rispetto a $[x_m + 1, h]$.

Esempio 1.6.1 (Esempio di Decomposizione Ricorsiva dalle Slide). Si consideri la lista di $n = 12$ identificatori compresi nell'intervallo $[0, 62]$:

$$L = [3, 4, 7, 13, 14, 15, 21, 25, 36, 38, 54, 62] \quad (n = 12, m = 6, l = 0, h = 62)$$

L'albero di decomposizione partiziona ricorsivamente la sequenza:

- L'elemento centrale divide la sequenza nelle sottoliste $[3, 4, 7, 13, 14]$ con vincolo $(n = 5, m = 3, l = 0, h = 14)$ e $[21, 25, 36, 38, 54]$ con vincolo $(n = 5, m = 3, l = 16, h = 62)$.
- A livello inferiore, la prima sottolista si divide in $[3, 4]$ con parametri $(n = 2, m = 1, l = 0, h = 6)$ e $[13, 14]$ con parametri $(n = 2, m = 1, l = 8, h = 14)$.
- Le foglie codificano i singoli elementi: ad esempio $[14]$ con parametri $(n = 1, m = 1, l = 14, h = 14)$, per cui $R = 14 - 14 - 1 + 1 = 1$, richiedendo esattamente $\lceil \log_2 1 \rceil = 0$ bit!

Quando i documenti sono densamente aggregati (*clusterizzati*), il numero di valori ammissibili R crolla verso 1, portando il costo di memorizzazione a zero bit per molti elementi.

1.6.2 Compressione Basata su Dizionario: LZ77

Per testi generali o stream di token, l'algoritmo **Lempel-Ziv '77** (LZ77) comprime la sequenza tramite una finestra scorrevole divisa in un buffer di ricerca (*Search Buffer*) e un buffer di anteprima (*Lookahead Buffer*), emettendo triple formattate come:

$$(\text{offset}, \text{length}, \text{next_char}) \quad (1.5)$$

Ad esempio, data la sequenza testuale:

$$T = \text{a b a b a a a b a b a a a a b}$$

La codifica LZ77 emette la successione di puntatori:

$$(0, 'a'), (0, 'b'), (2, 3), (1, 3), (7, 8)$$

sostituendo lunghe ripetizioni lessicali con riferimenti compatti all'indietro nella finestra.

1.7 Evoluzione verso il Neural Information Retrieval

A partire dal 2018, l'avvento dei modelli basati su architettura Transformer e di BERT ha rivoluzionato il panorama dell'Information Retrieval [4, 5]. I moderni approcci neurali si suddividono in due famiglie architetture fondamentali:

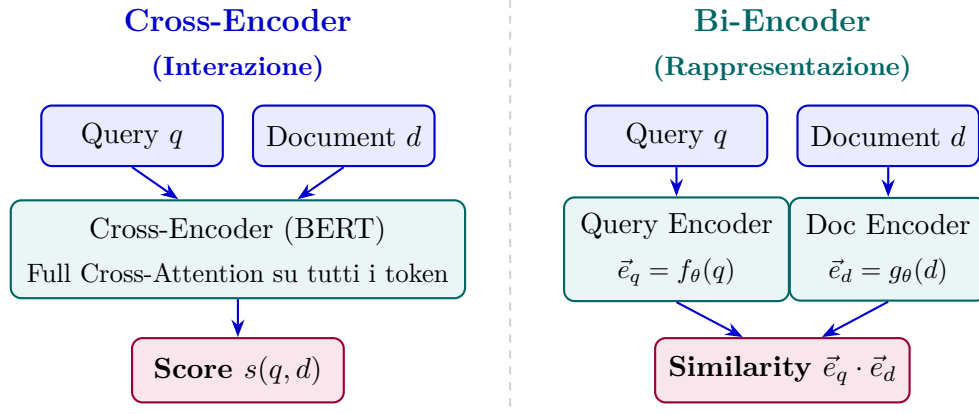


Figura 1.3: Confronto strutturale tra Cross-Encoder (Interazione) e Bi-Encoder (Rappresentazione).

1. **Modelli Basati sull'Interazione (*Cross-Encoders*):** La query e il documento vengono concatenati in un unico input:

$$\text{Input} = [\text{CLS}] \circ q \circ [\text{SEP}] \circ d$$

ed elaborati congiuntamente attraverso tutti i livelli di self-attention del Transformer. Catturano interazioni contestuali profonde tra ciascuna coppia di token, offrendo la massima efficacia predittiva. Tuttavia, il

costo computazionale proibitivo ($\mathcal{O}(|\mathcal{D}| \times \text{Transformer})$) ne preclude l'uso su larga scala; vengono impiegati esclusivamente come re-ranker di secondo livello sui top-100 candidati.

2. **Modelli Basati sulla Rappresentazione (*Bi-Encoders*)**: La query e il documento vengono proiettati in modo disaccoppiato in uno spazio vettoriale mediante due encoder distinti. I vettori dei documenti possono essere pre-calcolati offline e indicizzati, riducendo il matching online a un prodotto scalare o a una ricerca di vicini approssimati.

1.7.1 Le Tre Famiglie di Rappresentazioni Vettoriali

Nel contesto dei modelli a rappresentazione, si distinguono tre tipologie di embedding:

1. **Dense Single-Vector (es. DPR, Contriever)**: Ogni documento e ogni query sono sintetizzati in un unico vettore denso a d dimensioni (ad es. $d = 768$). L'interrogazione richiede indici vettoriali k-ANN su grafi (come HNSW).
2. **Dense Multi-Vector (es. ColBERT [4])**: Viene generato un vettore denso per ciascun token del documento e della query. La rilevanza è determinata tramite il meccanismo di **Late Interaction** con operatore *MaxSim*:

$$S(q, d) = \sum_{t_q \in q} \max_{t_d \in d} (\vec{E}_{t_q} \cdot \vec{E}_{t_d}^T) \quad (1.6)$$

Preserva l'identità dei singoli termini offrendo un'eccellente generalizzazione, a fronte di un costo di memorizzazione più consistente.

3. **Sparse Single-Vector (es. SPLADE [5])**: I documenti e le query sono codificati in vettori sparsi ad altissima dimensione mappati direttamente sui token del vocabolario $|V|$ ($|V| \approx 30\,000$). I coefficienti numerici indicano il peso d'importanza appreso e includono l'espansione automatica di sinonimi.
 - **Vantaggi**: Altissima efficacia sia nelle collezioni di dominio (*in-domain*) sia nei test a trasferimento zero-shot (*out-of-domain*).
 - **Efficienza**: Più compatti e rapidi rispetto ai modelli multi-vettore, possono essere memorizzati direttamente all'interno delle posting list di un classico Inverted Index.
 - **Interpretabilità**: I pesi non nulli corrispondono a parole comprensibili dall'uomo, facilitando il debug e la spiegabilità.

1.8 Ricerca dei Vicini Approssimati (k -ANN)

Nei sistemi vettoriali ad alta dimensionalità, il problema fondamentale consiste nel reperire i primi k documenti con il massimo prodotto scalare rispetto alla query (*Maximum Inner Product Search*, MIPS):

$$\operatorname{argmax}_{v \in \mathcal{D}}^{(k)} \vec{q}^T \cdot \vec{v} \quad (1.7)$$

In una tipica collezione Web su larga scala:

- Il numero complessivo di documenti supera gli 8.8 milioni (ad es. benchmark MS MARCO).
- Lo spazio vettoriale comprende circa 30 000 componenti lessicali distinte.
- Ciascun documento contiene in media solo circa 150 componenti con valore non nullo.

Per eseguire ricerche a frequenze di migliaia di query per secondo senza scansione esaustiva lineare, vengono utilizzate strutture dati specializzate che combinano indici diretti (*Forward Index*) e indici invertiti sparsi ottimizzati, quali la libreria ad alte prestazioni **TusKANNy** sviluppata in linguaggio Rust presso l'Università di Pisa.

Capitolo 2

Valutazione dei Sistemi di Information Retrieval

2.1 Come Valutare la Qualità di un Motore di Ricerca

La valutazione rigorosa delle prestazioni rappresenta una componente imprescindibile nella progettazione e nell'ottimizzazione dei sistemi di Information Retrieval [1, 2]. L'analisi delle prestazioni investe due dimensioni tra loro complementari:

1. Metriche di Efficienza:

- *Velocità di indicizzazione*: Quanti documenti per ora il sistema è in grado di processare e inserire nell'indice.
- *Indicizzazione incrementale*: Capacità di aggiornare l'indice con nuovi flussi documentali (ad es. +10 000 prodotti al giorno in un e-commerce) senza interruzioni di servizio.
- *Velocità di ricerca (Latenza e Throughput)*: Tempo di risposta (latenza al 95° o 99° percentile) e consumo di CPU/memoria nell'elaborazione di migliaia di query al secondo su collezioni di svariati milioni o miliardi di documenti.

2. Metriche di Efficacia e Qualità dei Servizi Accessori:

- Qualità delle raccomandazioni correlate, accuratezza della correzione ortografica delle query (*spell checking*) e chiarezza dei riassunti testuali visualizzati (*snippet generation*).

- **Efficacia intrinseca della graduatoria:** Misura la capacità fondamentale del motore di ricerca di soddisfare l'utente, ordinando i documenti pertinenti nelle prime posizioni della pagina dei risultati (*Search Engine Results Page*, SERP).

2.2 Misurare la Soddisfazione dell'Utente

Nel contesto operativo dei motori di ricerca commerciali, l'obiettivo ultimo è massimizzare la soddisfazione dell'utente (*User Happiness*). Poiché la felicità è uno stato cognitivo non osservabile direttamente, i sistemi di produzione ricorrono a proxy comportamentali online:

- **Clic e Click-Through Rate (CTR):** Il numero e la proporzione di clic ricevuti dai risultati restituiti.
 - *Criticità:* Titoli sensazionalistici o riassunti ingannevoli (*clickbait*) inducono clic elevati senza che il documento soddisfi realmente l'utente.
 - *La ricerca a zero clic (Zero-Click Search):* L'assenza di clic non è necessariamente un segnale negativo. Quando un utente cerca informazioni fattuali (ad es. il meteo, l'orario di un volo, una conversione di valuta o una definizione enciclopedica), la risposta visualizzata direttamente tramite *Featured Snippet* o *Knowledge Graph* soddisfa istantaneamente il bisogno senza richiedere alcun clic.
- **Conversioni e Transazioni Economiche:** Negli e-commerce, la quota di sessioni di ricerca che sfociano nell'acquisto di un prodotto e l'ammontare monetario speso dall'utente costituiscono metriche primarie di business.
- **Fidelizzazione e Visite Ripetute:** La percentuale di utenti che tornano ad adoperare il motore di ricerca entro un intervallo temporale predefinito (una settimana, un mese).
- **Dwell Time (Tempo di Permanenza):** L'intervallo di tempo che intercorre tra il momento in cui l'utente clicca su un risultato della SERP e l'istante in cui fa ritorno alla pagina dei risultati (*bouncing back*). Un Dwell Time prolungato indica generalmente che il documento ha catturato l'attenzione ed è risultato informativo, mentre un ritorno immediato alla SERP denota insoddisfazione.

La Valutazione nell'Era dei Motori Generativi e LLM

L'emergere di paradigmi basati su modelli linguistici (come ChatGPT, SearchGPT e Perplexity) ridefinisce radicalmente il concetto di Dwell Time e CTR: l'utente non naviga più attraverso link esterni, ma consuma sintesi testuali generate direttamente. In tale scenario, l'accuratezza fattuale, l'assenza di allucinazioni e l'attribuzione corretta delle fonti rimpiazzano il conteggio dei clic tradizionali.

2.3 Il Paradigma di Cranfield e le Collezioni Benchmark

Sebbene i segnali online forniscano un feedback prezioso, essi risultano rumorosi, soggetti a forti bias di posizionamento (*Position Bias*) e non riproducibili sperimentalmente prima del rilascio in produzione. Per consentire valutazioni comparative controllate, la ricerca in IR adotta il **Paradigma di Cranfield**, sviluppato originariamente da Cyril Cleverdon negli anni '60 [6].

Definizione 2.3.1 (Collezione di Test (Test Collection)). Una collezione di test standard per l'Information Retrieval è una tripletta formale $\mathcal{T} = (\mathcal{D}, \mathcal{Q}, \mathcal{J})$ costituita da:

1. Una collezione documentale benchmark $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$;
2. Una suite benchmark di bisogni informativi e query $\mathcal{Q} = \{q_1, q_2, \dots, q_M\}$;
3. Un insieme di giudizi di rilevanza (*Relevance Judgments* o *qrels*) $\mathcal{J} = \{(q, d) \mapsto r_{q,d}\}$, che associa a ciascuna coppia query-documento un valore di rilevanza (binario o graduato).

2.3.1 La Sfida della Scalabilità Combinatoria

La realizzazione manuale esaustiva dei giudizi di rilevanza è materialmente impossibile per archivi realistici. Si consideri uno scenario benchmark con:

$$|\mathcal{D}| = 5 \times 10^6 \text{ documenti}, \quad |\mathcal{Q}| = 50\,000 \text{ query}$$

Lo spazio complessivo delle coppie ammonta a:

$$|\mathcal{D}| \times |\mathcal{Q}| = 5 \times 10^6 \times 5 \times 10^4 = 2.5 \times 10^{11} \text{ coppie query-documento}$$

Ipotizzando che un annotatore umano impieghi in media 2.5 secondi per valutare la rilevanza di un documento:

$$\begin{aligned}\text{Tempo Totale} &= 2.5 \times 10^{11} \times 2.5 \text{ s} = 6.25 \times 10^{11} \text{ secondi} \\ &\approx 173\,611\,111 \text{ ore/persona}\end{aligned}$$

Un impegno titanico che richiederebbe decine di migliaia di anni di lavoro umano ininterrotto! Per ovviare a ciò, i benchmark limitano i giudizi a un sottoinsieme accuratamente campionato di documenti.

2.3.2 Crowdsourcing: Opportunità e Limiti

Una strategia economica consiste nel delegare la valutazione a lavoratori retribuiti su piattaforme online di crowdsourcing (come Amazon Mechanical Turk). Tuttavia, la letteratura scientifica evidenzia come tali giudizi siano affetti da:

- Elevatissima varianza statistica;
- Minore competenza e presenza di spammer;
- Necessità di filtri rigorosi di consenso (ad es. votazione di maggioranza o calcolo dell'accordo inter-annotatore tramite coefficiente κ di Cohen).

2.3.3 L'Infrastruttura TREC (*Text REtrieval Conference*)

Promossa dal National Institute of Standards and Technology (NIST), la conferenza **TREC** rappresenta il punto di riferimento internazionale per la valutazione delle tecnologie di recupero dell'informazione. TREC è strutturata in tracce tematiche (*Tracks*), ciascuna incentrata su specifici casi d'uso (ricerca Web, ricerca clinica, domande-risposte, deep retrieval). Il risultato prodotto da un sistema software che esegue un compito su una collezione di test prende il nome di **Run**.

2.3.4 La Tecnica del Pooling

Per campionare i documenti da sottoporre ad annotazione umana senza dover valutare l'intera collezione, nei benchmark TREC si adotta la tecnica del **Pooling** (Figura 2.1):

1. Si raccolgono le risposte fornite da m sistemi di recupero eterogenei per una determinata query.

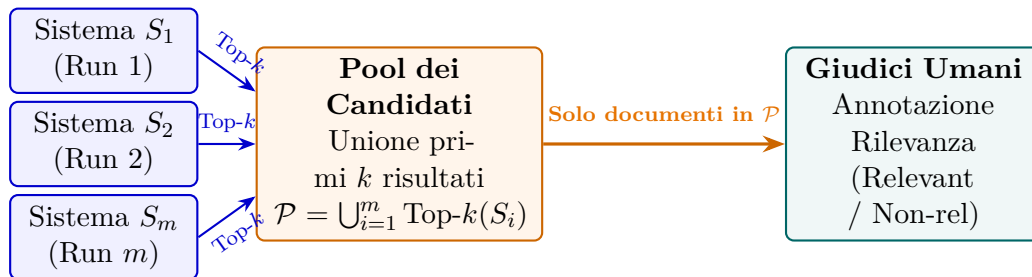


Figura 2.1: Processo di Pooling per la selezione controllata dei documenti da annotare.

2. Si estraggono i primi k documenti restituiti da ciascun sistema (tipicamente $k = 50$ o $k = 100$).
3. Si calcola l'**unione** di tali insiemi eliminando i duplicati, formando il pool $\mathcal{P}$.
4. Gli annotatori umani valutano la rilevanza unicamente per i documenti appartenenti a $\mathcal{P}$.

L'Assunzione Fondamentale del Pooling

Tutti i documenti dell'archivio che non compaiono nel pool (*unpooled*) vengono assunti convenzionalmente come **non rilevanti** ($r = 0$). Sebbene tale assunzione introduca un potenziale bias a sfavore di algoritmi futuri con paradigmi radicalmente differenti, studi empirici condotti da NIST confermano che, a fronte di un pool formato da sistemi sufficientemente diversificati, le conclusioni comparative tra motori di ricerca rimangono stabili e affidabili.

2.4 Bisogno Informativo vs. Query

Una distinzione concettuale decisiva riguarda il divario tra bisogno informativo e formulazione della query:

- **Bisogno Informativo (*Information Need*):** Rappresenta lo stato cognitivo e l'obiettivo reale dell'utente. Ad esempio: *"Il fondo della mia piscina sta diventando nero e necessita di essere ripulito"*.
- **Query:** L'espressione testuale sintetica digitata nel box di ricerca. Ad esempio: *"pulitore piscina"*.

Proprietà 2.4.1 (Principio di Valutazione della Rilevanza). *La rilevanza di un documento deve essere sempre stimata rispetto all'**Information Need** sottostante, mai rispetto alla mera sequenza di caratteri che compone la query.*

2.5 Metriche di Valutazione Non Ordinate (*Unranked Measures*)

Nel contesto del reperimento non ordinato (come nel modello booleano classico), il sistema restituisce un insieme non graduato di documenti. La valutazione si fonda sulla suddivisione dell'universo documentale mediante la matrice di contingenza (Tabella 2.1):

	Documento Rilevante	Documento Non Rilevante
Documento Reperito	True Positive (TP)	False Positive (FP)
Documento Non Reperito	False Negative (FN)	True Negative (TN)

Tabella 2.1: Matrice di contingenza per la classificazione binaria query-documento.

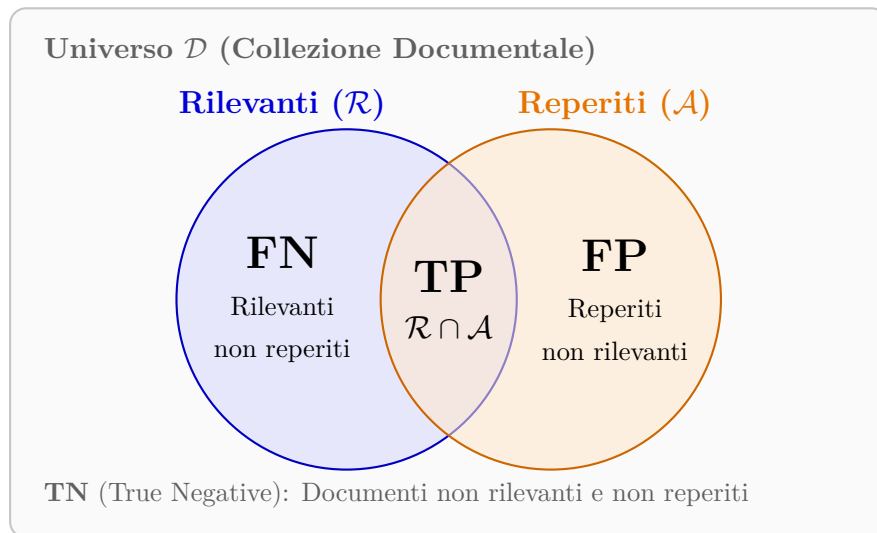


Figura 2.2: Rappresentazione insiemistica del reperimento documentale e delle regioni della matrice di confusione.

2.5.1 Precisione, Richiamo e loro Trade-off

Definizione 2.5.1 (Precisione (*Precision*, P)). La frazione dei documenti restituiti dal sistema che risultano effettivamente rilevanti:

$$P = \frac{TP}{TP + FP} = P(\text{rilevante} \mid \text{reperito}) \quad (2.1)$$

Definizione 2.5.2 (Richiamo (*Recall*, R)). La frazione dei documenti rilevanti complessivamente presenti nella collezione che il sistema è riuscito a reperire:

$$R = \frac{TP}{TP + FN} = P(\text{reperito} \mid \text{rilevante}) \quad (2.2)$$

Esiste un intrinseco **trade-off** tra Precisione e Richiamo:

- Un sistema che restituisce l'intera collezione documentale (Retrieved = $\mathcal{D}$) ottiene banamente un richiamo perfetto $R = 100\%$, ma la sua precisione crolla a livelli minimi ($P \approx 0$).
- Viceversa, un sistema conservativo che restituisce unicamente il singolo documento più sicuro può conseguire $P = 100\%$, pagando il prezzo di un richiamo trascurabile.

2.5.2 La Media Armonica: F-Measure (F_1 -Score)

Per sintetizzare Precisione e Richiamo in un unico valore scalare che premi l'equilibrio tra le due grandezze, si adotta la **F-Measure** (o F_1 -score), definita come la **media armonica** di P ed R :

$$F_1 = \frac{2}{\frac{1}{P} + \frac{1}{R}} = \frac{2 \cdot P \cdot R}{P + R} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN} \quad (2.3)$$

Proprietà 2.5.1 (Proprietà della Media Armonica). *La media armonica gode di proprietà matematiche fondamentali per la valutazione:*

1. *È sempre minore o uguale alla media geometrica e alla media aritmetica:*

$$\frac{2}{\frac{1}{a} + \frac{1}{b}} \leq \sqrt{a \cdot b} \leq \frac{a + b}{2}$$

2. *Quando i due argomenti differiscono notevolmente, la media armonica collapsa verso il **valore minimo** anziché posizionarsi a metà strada.*

Esempio 2.5.1 (Comportamento Limite dell'F-Measure). Si consideri un sistema banale che restituisce l'intera collezione: $R = 1.0$ e $P = 0.0001$.

- La media aritmetica assegna un punteggio fuorviante pari a:

$$\frac{1.0 + 0.0001}{2} \approx 0.50 \quad (50\%)$$

- La media armonica (F_1) demolisce giustamente la valutazione:

$$F_1 = \frac{2 \cdot 1.0 \cdot 0.0001}{1.0 + 0.0001} \approx 0.0002 \quad (0.02\%)$$

Ciò dimostra per quale motivo la media aritmetica sia inadeguata e l’F-measure costituisca la scelta corretta.

2.5.3 La Misura F_β Pesata

Qualora si desideri attribuire pesi differenziati a Precisione e Richiamo, si introduce un parametro di ponderazione $\alpha \in [0, 1]$:

$$F = \frac{1}{\alpha \frac{1}{P} + (1 - \alpha) \frac{1}{R}} \quad (2.4)$$

Esprimendo α in funzione di un parametro β tale che $\alpha = \frac{1}{1+\beta^2}$, si ricava la formulazione classica di F_β :

$$F_\beta = \frac{(1 + \beta^2) \cdot P \cdot R}{\beta^2 \cdot P + R} \quad (2.5)$$

- Valori di $\beta < 1$ (ad es. $\beta = 0.5$) enfatizzano la **Precisione** rispetto al Richiamo (utile nelle ricerche su Web generico).
- Valori di $\beta > 1$ (ad es. $\beta = 2.0$) enfatizzano il **Richiamo** rispetto alla Precisione (fondamentale nelle ricerche brevettuali, legali e mediche, dove omettere un documento rilevante comporta danni gravi).

2.6 Metriche Basate sul Ranking (*Ranked Measures* - Rilevanza Binaria)

Nei motori di ricerca moderni i documenti non vengono restituiti come insiemi informi, ma ordinati in base a uno score di rilevanza decrescente. Inoltre, gli utenti Web raramente oltrepassano la prima pagina dei risultati (i primi 10 o 20 elementi).

2.6.1 Precision@k ($P@k$) e Recall@k ($R@k$)

Definizione 2.6.1 (Precision@k). Fissata una soglia di rango $k \in \mathbb{N}^+$, la Precision@k corrisponde alla proporzione di documenti rilevanti presenti nei primi k posti della graduatoria:

$$P@k = \frac{1}{k} \sum_{i=1}^k \text{rel}_i \quad (2.6)$$

dove $\text{rel}_i \in \{0, 1\}$ denota la rilevanza binaria del documento al rango i .

Esempio 2.6.1 (Calcolo di Precision@k). Si consideri una lista ordinata in cui la sequenza di rilevanza binaria dei primi 5 documenti è data da $\langle 1, 1, 0, 0, 1 \rangle$:

- $P@1 = \frac{1}{1} = 1.00$
- $P@2 = \frac{2}{2} = 1.00$
- $P@3 = \frac{2}{3} \approx 0.66$
- $P@4 = \frac{2}{4} = 0.50$ (si noti che $P@4 < P@3$)
- $P@5 = \frac{3}{5} = 0.60$

Limiti di $P@k$: Non tiene conto della posizione interna dei documenti entro la soglia k (un rilevante al rango 1 o al rango k contribuisce nello stesso modo) e non si presta a medie stabili tra query che possiedono un numero totale di rilevanti inferiore a k . In modo analogo, $\text{Recall}@k$ ($R@k$) calcola la frazione dei documenti rilevanti complessivi reperiti entro il rango k .

2.6.2 Average Precision (AP)

L'**Average Precision** integra la precisione calcolata a ciascun rango in cui compare un documento rilevante, premiando i sistemi che posizionano i documenti utili più in alto nella SERP:

Definizione 2.6.2 (Average Precision). Dato un ordinamento di N documenti per una query avente R documenti rilevanti complessivi nella collezione, l'AP è definita come:

$$AP = \frac{1}{R} \sum_{k=1}^N P@k \cdot \text{rel}_k \quad (2.7)$$

Esempio 2.6.2 (Calcolo Dettagliato dell'AP dalle Slide). Si consideri una query per la quale esistono $R = 3$ documenti rilevanti totali. Il motore di

ricerca restituisce tali documenti ai ranghi $k_1 = 1$, $k_2 = 3$ e $k_3 = 5$. Le precisioni corrispondenti sono:

$$P@1 = \frac{1}{1} = 1.00, \quad P@3 = \frac{2}{3} \approx 0.667, \quad P@5 = \frac{3}{5} = 0.60$$

L’Average Precision risulta pari a:

$$AP = \frac{1}{3} \left(1 + \frac{2}{3} + \frac{3}{5} \right) = \frac{1}{3} (1.00 + 0.667 + 0.600) = \frac{2.267}{3} \approx 0.756 \quad (2.8)$$

2.6.3 Mean Average Precision (MAP)

Quando si valuta l’efficacia globale su una suite di $|\mathcal{Q}|$ query benchmark, si calcola la media aritmetica dei singoli valori di AP:

Definizione 2.6.3 (Mean Average Precision).

$$MAP = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} AP(q) \quad (2.9)$$

Proprietà 2.6.1 (Macro-Averaging di MAP). *Il MAP adotta una strategia di **macro-averaging**: ogni query (ogni information need) possiede lo stesso peso statistico pari a $1/|\mathcal{Q}|$, indipendentemente dal fatto che una query possieda 2 documenti rilevanti e un’altra ne conti 200. Rappresenta la metrica standard più accreditata per la rilevanza binaria nella letteratura scientifica.*

2.6.4 Mean Reciprocal Rank (MRR)

In scenari quali le ricerche di elementi noti (*Known-Item Search*), le query navigazionali (ad es. cercare l’home page di un ateneo) o il reperimento di risposte fattuali, l’utente è interessato unicamente a individuare la **prima risposta corretta**. Lo sforzo cognitivo e il tempo speso dall’utente sono inversamente proporzionali alla posizione occupata dal primo risultato rilevante:

Definizione 2.6.4 (Reciprocal Rank e MRR). Sia $\text{rank}_1(q)$ la posizione del primo documento rilevante restituito per la query q . Il **Reciprocal Rank** (RR) è dato da:

$$RR(q) = \frac{1}{\text{rank}_1(q)} \quad (2.10)$$

Il **Mean Reciprocal Rank** (MRR) è la media aritmetica su tutte le query:

$$MRR = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \frac{1}{\text{rank}_1(q)} \quad (2.11)$$

Se per una query il primo documento rilevante si trova al rango 1, il punteggio è 1.0; se scende al rango 2 diviene 0.5; se scende al rango 10 crolla a 0.1.

2.7 Oltre la Rilevanza Binaria: Rilevanza Graduata e NDCG

Nelle applicazioni Web e commerciali, il modello binario (rilevante/non rilevante) si rivela limitante. Gli utenti percepiscono gradi sfumati di utilità, tipicamente quantificati su scale discrete (ad esempio da 0 a 3, dove 0 = Non Rilevante, 1 = Marginale, 2 = Rilevante, 3 = Altamente Rilevante).

L'approccio del **Discounted Cumulative Gain** (DCG), introdotto da Järvelin e Kekäläinen [7], poggia su due assunzioni fondamentali:

1. I documenti altamente rilevanti apportano un'utilità (*gain*) di gran lunga superiore rispetto a documenti solo marginalmente rilevanti.
2. Minore è la posizione ordinale (rango) di un documento nella graduatoria, minore è la probabilità che l'utente lo esamini; pertanto, il guadagno deve essere progressivamente scontato (*discounted*) verso il basso.

2.7.1 Cumulative Gain (CG) e Discounted Cumulative Gain (DCG)

Definizione 2.7.1 (Cumulative Gain a Rango p). Dato il vettore delle rilevanze graduate $\mathbf{r} = [r_1, r_2, \dots, r_p]$, il Cumulative Gain è la somma lineare dei guadagni:

$$CG_p = \sum_{i=1}^p r_i \quad (2.12)$$

Il CG_p è insensibile alla posizione: scambiare un documento altamente rilevante al rango 1 con uno al rango 10 non altera il valore di CG_p .

Definizione 2.7.2 (Discounted Cumulative Gain (Formulazione Standard)). Il DCG applica uno sconto logaritmico basato sul rango della posizione i :

$$DCG_p = r_1 + \sum_{i=2}^p \frac{r_i}{\log_2(i)} \quad (2.13)$$

Nei motori di ricerca commerciali (Google, Bing) si impiega frequentemente la **formulazione esponenziale**:

$$DCG_p^{\text{exp}} = \sum_{i=1}^p \frac{2^{r_i} - 1}{\log_2(i + 1)} \quad (2.14)$$

Tale variante accentua in modo esponenziale il valore dei documenti con alta rilevanza ($r = 3 \implies 2^3 - 1 = 7$, contro $r = 1 \implies 2^1 - 1 = 1$), penalizzando severamente il loro eventuale declassamento.

2.7.2 Normalized Discounted Cumulative Gain (NDCG)

Poiché query differenti presentano un numero variabile di documenti rilevanti, il valore grezzo di DCG_p non è confrontabile tra query diverse. Si ricorre quindi all'**Ideal DCG** ($IDCG_p$), ottenuto calcolando il DCG della graduatoria ideale che ordina tutti i documenti per rilevanza decrescente:

$$NDCG_p = \frac{DCG_p}{IDCG_p} \in [0, 1] \quad (2.15)$$

2.7.3 Esempio Numerico Completo: Calcolo Passo-Passo

Si consideri la graduatoria di 10 documenti tratta dalle slide del corso, valutati su scala 0–3:

$$\mathbf{r} = [3, 2, 3, 0, 0, 1, 2, 2, 3, 0]$$

La sequenza ideale ottenuta ordinando per rilevanza non crescente è:

$$\mathbf{r}^* = [3, 3, 3, 2, 2, 2, 1, 0, 0, 0]$$

La Tabella 2.2 riporta l'evoluzione dettagliata di ogni singolo termine posizionale.

i	r_i	Sconto	Guadagno Scontato	$DCG@i$	r_i^*	Guadagno Ideale	$IDCG@i$	$NDCG@i$
1	3	1.000	3.00	3.00	3	3.00	3.00	1.00
2	2	$\log_2(2) = 1.000$	2.00	5.00	3	3.00	6.00	0.83
3	3	$\log_2(3) = 1.585$	1.89	6.89	3	1.89	7.89	0.87
4	0	$\log_2(4) = 2.000$	0.00	6.89	2	1.00	8.89	0.76
5	0	$\log_2(5) = 2.322$	0.00	6.89	2	0.86	9.75	0.71
6	1	$\log_2(6) = 2.585$	0.39	7.28	2	0.77	10.52	0.69
7	2	$\log_2(7) = 2.807$	0.71	7.99	1	0.36	10.88	0.73
8	2	$\log_2(8) = 3.000$	0.67	8.66	0	0.00	10.88	0.80
9	3	$\log_2(9) = 3.170$	0.95	9.61	0	0.00	10.88	0.88
10	0	$\log_2(10) = 3.322$	0.00	9.61	0	0.00	10.88	0.88

Tabella 2.2: Tracciamento dettagliato del calcolo di $DCG@i$, $IDCG@i$ e $NDCG@i$ per $i = 1, \dots, 10$.

2.7.4 Confronto tra Funzioni di Ranking Alternative

Si consideri il caso studio di 4 documenti $\{d_1, d_2, d_3, d_4\}$ con gradi di rilevanza:

$$r(d_1) = 0, \quad r(d_2) = 1, \quad r(d_3) = 2, \quad r(d_4) = 2$$

- **Ground Truth (GT):** Ordinamento $\langle d_4, d_3, d_2, d_1 \rangle$ con vettori di rilevanza $[2, 2, 1, 0]$:

$$IDCG_4 = 2 + \frac{2}{\log_2(2)} + \frac{1}{\log_2(3)} + 0 = 2 + 2 + 0.6309 = 4.6309$$

$$NDCG_{GT} = 1.00$$

- **Ranking Function 1 (RF_1):** Ordinamento $\langle d_3, d_4, d_2, d_1 \rangle$ con rilevanze $[2, 2, 1, 0]$. Poiché i primi due documenti possiedono il medesimo grado ($r = 2$), l'inversione non produce alcuna penalità posizionale:

$$DCG_{RF_1} = 2 + \frac{2}{\log_2(2)} + \frac{1}{\log_2(3)} = 4.6309$$

$$NDCG_{RF_1} = \frac{4.6309}{4.6309} = 1.00$$

- **Ranking Function 2 (RF_2):** Ordinamento $\langle d_3, d_2, d_4, d_1 \rangle$ con rilevanze $[2, 1, 2, 0]$. In questo caso, il documento d_2 ($r = 1$) scavalca erroneamente d_4 ($r = 2$):

$$DCG_{RF_2} = 2 + \frac{1}{\log_2(2)} + \frac{2}{\log_2(3)} + 0 = 2 + 1 + \frac{2}{1.585} = 4.2619$$

$$NDCG_{RF_2} = \frac{DCG_{RF_2}}{IDCG_4} = \frac{4.2619}{4.6309} \approx 0.9203$$

Questo calcolo dimostra con precisione matematica come la metrica NDCG penalizzi le inversioni di rango proporzionalmente alla gravità dello scambio di rilevanza.

Bibliografia

- [1] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [2] Stefan Büttcher, Charles L. A. Clarke, and Gordon V. Cormack. *Information Retrieval: Implementing and Evaluating Search Engines*. MIT Press, 2010.
- [3] Alistair Moffat and Luke Stuiver. Binary interpolative coding for effective index compression. *Information Processing & Management*, 36(5):667–687, 2000.
- [4] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020.
- [5] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse lexical and expansion model for information retrieval. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2288–2292, 2021.
- [6] Cyril W. Cleverdon. The Cranfield tests on index language devices. *Aslib Proceedings*, 19(6):173–194, 1967.
- [7] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of retrieval techniques. *ACM Transactions on Information Systems (TOIS)*, 20(4):422–446, 2002.